CONFIDENTIAL · INTERNAL DOCUMENT

Master Tech Stack Specification

Complete architecture, infrastructure, and implementation spec for the Maase platform — India's hyperlocal home-cooked food marketplace.

PROJECT	VERSION	DATE	STATUS
Maase India	1.0.0	March 2026	Active Development

CONTACT

connect.maase@gmail.com

Contents

01 Project Overview 3

02 Monorepo Structure 3

03 Application Breakdown 4

04 Backend API 6

05 Database Schema 7

06 Authentication 8

07 Payments 8

08 Infrastructure & DevOps 9

09 Design System 10

10 Unique Product Features 11

11 Environment Variables 11

12 Development Setup 12

13 Launch Phases 13

01 Project Overview

What Maase is, and the problem it solves

Maase is India's hyperlocal marketplace for authentic home-cooked food. It connects home cooks — primarily homemakers — with nearby customers who want fresh, daily home-cooked meals within a 3km radius. No dark kitchens, no restaurants. Real cooking from real homes.

🍱 **Core insight:** Indian kitchens are already active every day. Gas is on, ingredients are ready, cooking is happening for the family. One extra portion = almost no extra effort, but an entirely new income stream for the household.

🏠

Home Cooks

Homemakers earn from existing daily cooking

Supplier

🛵

Delivery

3km hyperlocal delivery partners handle logistics

Operations

📱

Customers

Neighbours wanting fresh authentic daily meals

Consumer

📍

Bangalore First

Pilot launch — Koramangala, HSR, Whitefield, BTM

Phase 1

02 Monorepo Structure

Turborepo + pnpm workspaces

```
maase/
├── src/                    # Customer mobile app (Expo, root level)
│   ├── screens/            # 11 screens (Login → Track)
│   ├── components/         # Shared UI components
│   ├── constants/          # theme.ts — Colors, Typography, Spacing
│   ├── hooks/
│   ├── navigation/
│   └── utils/              # firebase.ts, supabase.ts
├── apps/
│   ├── cook-dashboard/     # Cook web dashboard (Next.js 14)
│   ├── admin/              # Internal admin panel
│   └── web/               # Public marketing site — maaseindia.com
├── services/
│   └── api/               # REST API — Express + Prisma (Railway)
│       ├── src/routes/     # cooks, meals, orders, payments, ...
│       └── prisma/         # schema.prisma
├── packages/
│   ├── types/             # Shared TypeScript types
│   ├── config/            # Shared ESLint, TS config
│   └── ui/               # Shared UI components
├── supabase/
│   └── schema.sql          # Full DB schema
├── web-preview/           # Static landing page (Vercel)
│   ├── index.html           # Marketing + chef onboarding
│   ├── chef-profile.html    # Shareable chef profile
│   └── supabase-schema.sql  # Pre-launch tables
├── turbo.json
└── pnpm-workspace.yaml
```

Turborepo | pnpm workspaces | TypeScript throughout | Shared packages

03 Application Breakdown

5 distinct apps, each with its own deployment

3.1 Customer Mobile App

The primary consumer-facing product. Available on iOS and Android via Expo. Bundle ID: `com.maase.customerapp`

ITEM	DETAIL
Framework	React Native 0.81 + Expo SDK 54
Language	TypeScript 5.3
Navigation	React Navigation 6 — NativeStack + BottomTabs
Auth	Firebase Auth — Phone OTP (primary) + Google OAuth
Database Client	Supabase JS v2
Fonts	Playfair Display + Poppins via expo-google-fonts
Location	expo-location
Local Storage	@react-native-async-storage/async-storage
Web Support	react-native-web (expo web build)
Build Tool	Expo EAS Build (iOS + Android)

Screens (11 total)

Onboarding

SplashScreen, LoginScreen (OTP + Google), LocationScreen

Browse & Order

HomeScreen, MealDetailScreen, CartScreen, PaymentScreen

Post-Order

OrderConfirmationScreen, OrderTrackingScreen, OrdersScreen, ProfileScreen

3.2 Cook Dashboard

Web interface for home cooks to manage daily menu, view orders, mark batches ready, and track earnings.

ITEM	DETAIL
Framework	Next.js 14 (App Router)
Language	TypeScript 5.3
Styling	Tailwind CSS 3.4
Database	Supabase JS v2
Deploy	Vercel

3.3 Admin Panel

Internal tool for the Maase ops team — cook verification, order oversight, metrics dashboard, dispute resolution.

3.4 Public Marketing Website

SEO-optimised public website at `maaseindia.com`. Next.js 14 + Tailwind CSS, deployed on Vercel.

3.5 Pre-Launch Landing Page

Live now at `web-preview-virid.vercel.app`. Single-file HTML — no build step, instant deploys via Vercel CLI.

ITEM	DETAIL
Stack	Pure HTML / CSS / Vanilla JS (single file)
Database	Supabase JS v2 via CDN
Fonts	Google Fonts CDN
Hosting	Vercel (CLI deploy, ~12s deploys)
Mobile	Fully responsive, iOS zoom fix, bottom-sheet modal

✅ **Live Supabase tables:** `chef_applications` (with referral codes), `chef_nominations`, `waitlist`. Row Level Security enabled. Real-time application count shown in hero.

---

04 Backend API

REST API — Express + Prisma, deployed on Railway

ITEM	DETAIL
Runtime	Node.js + TypeScript
Framework	Express 4
ORM	Prisma 5 (PostgreSQL)
Validation	Zod
Auth	JWT (jsonwebtoken)
Payments	Razorpay Node SDK
Security	Helmet + CORS
Deploy	Railway (Nixpacks, auto-restart on failure)
Health Check	GET /health

API Route Groups

ROUTE PREFIX	FILE	PURPOSE
/api/profiles	profiles.routes.ts	Customer profiles, addresses, preferences
/api/cooks	cooks.routes.ts	Cook profiles, online status, locality
/api/meals	meals.routes.ts	Meal listings, batch management, availability
/api/orders	orders.routes.ts	Full order lifecycle (placed → delivered)
/api/payments	payments.routes.ts	Razorpay webhook + payment verification
/api/reviews	reviews.routes.ts	Ratings + comments
/api/donations	donations.routes.ts	Feed a Neighbour transactions

05 Database Schema

Supabase (PostgreSQL) — Project: aaixsytygsjilgvebnfv

Core Tables

TABLE	KEY COLUMNS	PURPOSE
profiles	id, phone, name, diet_preferences, addresses	Customer accounts linked to Firebase auth
cooks	id, name, cuisine, rating, locality, is_online, lat/lng, upi_id	Home cook profiles + payout info
meals	id, cook_id, name, price, batch_total, batch_remaining, cooked_at, eta_minutes	Daily meal listings with live batch tracking
orders	id (MASxxxx), user_id, meal_id, cook_id, quantity, status	Full order lifecycle
addresses	user_id, label, lat, lng, line1, locality	Customer delivery addresses
reviews	order_id, rating, comment, cook_id	Post-delivery star ratings
donations	donor_id, amount, locality, message	Feed a Neighbour transactions

Pre-Launch Tables

TABLE	KEY COLUMNS	PURPOSE
chef_applications	name, phone, area, cuisines[], referral_code, referred_by	Chef signup from landing page
chef_nominations	nominator_name, cook_name, cook_phone, reason	Community chef nominations
waitlist	phone, city, type	Customer early access list

🔒 **Row Level Security (RLS)** is enabled on all tables. Pre-launch tables allow anonymous insert + read. Production tables will use Supabase Auth JWTs for row-level access control.

ORM: Prisma schema at `services/api/prisma/schema.prisma` maps all models with snake_case ↔ camelCase.

06 Authentication

Multi-layer auth across apps

APP	METHOD	PROVIDER
Customer Mobile App	Phone OTP (primary) + Google OAuth	Firebase Auth
Cook Dashboard	Email + password	Supabase Auth
Admin Panel	Email + password + role check	Supabase Auth
API (service-to-service)	JWT Bearer tokens	jsonwebtoken

Firebase config: `google-services.json` (Android), `GoogleService-Info.plist` (iOS). OAuth client IDs stored in `EXPO_PUBLIC_GOOGLE_*` environment variables.

07

Payments

Razorpay — India's leading payment gateway

Order Payments

Razorpay SDK on mobile (UPI, cards, net banking). Server-side signature verification before order confirmation.

Webhooks

`POST /api/payments/webhook` — Razorpay notifies payment status. HMAC-SHA256 signature verification.

Cook Payouts

UPI ID or bank account stored on Cook model. Weekly settlement via Razorpay Route or manual transfer (Phase 1).

08 Infrastructure & DevOps

Fully managed cloud stack

COMPONENT	PROVIDER	NOTES
Customer App builds	Expo EAS Build	iOS + Android production builds
Landing page	Vercel	web-preview-virid.vercel.app
Marketing website	Vercel	maaseindia.com
Cook Dashboard	Vercel	Auto-deploy on push
Backend API	Railway	Nixpacks, health check /health, auto-restart
Database	Supabase	Hosted PostgreSQL, RLS, real-time
Auth	Firebase + Supabase Auth	Phone OTP via Firebase
File Storage	Supabase Storage	Meal photos, cook avatars
CDN	Supabase CDN	Assets served globally
Payments	Razorpay	INR, UPI, cards, net banking

🚀 **Zero-ops philosophy:** All infra is fully managed. No servers to maintain. Railway handles API scaling, Supabase handles DB, Vercel handles web deploys. The team focuses entirely on product.

09 Design System

Brand tokens, fonts, and mobile-first principles

Brand Fonts

FONT	USAGE	WEIGHTS
Playfair Display	Headlines, quotes, brand name	400, 700, italic
Poppins / Inter	Body text, labels, buttons	400, 500, 600, 700

Brand Colors

Turmeric `#F4A300`

Turmeric Deep `#D98D00`

Ivory `#F8F3E8`

Mocha `#5C3A21`

Mocha Soft `#7b573e`

Mobile Design Principles

- **Mobile-first** — majority of users on Android/iOS
- **Minimum touch target: 48px** — all buttons, inputs, chips
- **Input font-size ≥ 16px** — prevents iOS Safari auto-zoom on focus
- **Bottom-sheet modals** on mobile — native feel via CSS border-radius + align-items: flex-end
- **Tap highlight removed** — `-webkit-tap-highlight-color: transparent`
- **3km hyperlocal language** — never generic "India", always area-specific

10 Unique Product Features

What makes Maase different from Swiggy / Zomato

🟢

Cook's Kitchen Live

Real cook timing shown to customer — when food was started today, not an estimate. Complete transparency from kitchen to door.

Unique to Maase

🍳

Maa's Batch

True scarcity: if she makes 2 portions, only 2 can be sold. No dark patterns, no fake countdowns. Authenticity as a product feature.

Unique to Maase

🤲

Feed a Neighbour

One-tap ₹120 donation to feed someone nearby, anonymously. Community built into the product from day one.

Social Impact

🎁

Referral System

Each chef gets a unique code (e.g. SUNITA-7823) to invite other home cooks. Stored in Supabase, shareable via WhatsApp with pre-filled message.

Growth

👤

Shareable Chef Profile

Static public URL per chef. Built on URL params — no backend needed. Shareable as a WhatsApp link to attract customers pre-launch.

Growth

❤️

Nominate Your Mom

Community-driven chef onboarding. Anyone can nominate a home cook. Maase personally reaches out + sends welcome kit worth ₹500.

Community

11 Environment Variables

Required secrets per service — never commit to git

Customer App (.env / app.config.js)

EXPO_PUBLIC_SUPABASE_URL	Supabase project URL
EXPO_PUBLIC_SUPABASE_ANON_KEY	Supabase anon/public key
EXPO_PUBLIC_GOOGLE_WEB_CLIENT_ID	Firebase Google OAuth — web
EXPO_PUBLIC_GOOGLE_IOS_CLIENT_ID	Firebase Google OAuth — iOS
EXPO_PUBLIC_GOOGLE_ANDROID_CLIENT_ID	Firebase Google OAuth — Android
EXPO_PUBLIC_RAZORPAY_KEY_ID	Razorpay public key (client-safe)

API Service (.env)

DATABASE_URL	Supabase pooled connection (Prisma runtime)
DIRECT_URL	Supabase direct connection (migrations)
JWT_SECRET	Secret for signing API JWT tokens
RAZORPAY_KEY_ID	Razorpay key ID (server-side)
RAZORPAY_KEY_SECRET	Razorpay secret — never expose client-side
RAZORPAY_WEBHOOK_SECRET	For HMAC webhook signature verification
SUPABASE_SERVICE_ROLE_KEY	Admin key for server-side Supabase ops

12 Development Setup

How to run every service locally

Prerequisites

- Node.js 22+
- pnpm
- expo-cli
- vercel-cli
- Supabase CLI (optional)

```
# Install all workspace dependencies
pnpm install

# ── Customer App (physical device via tunnel) ──
./start-expo.sh
# or: nvm use 22 && npx expo start --tunnel

# ── API Service ──
cd services/api
npm run dev        # ts-node-dev, auto-restarts

# ── Cook Dashboard ──
cd apps/cook-dashboard
npm run dev        # Next.js dev server

# ── Marketing Website ──
cd apps/web
npm run dev

# ── Landing Page (instant deploy) ──
cd web-preview
npx vercel --prod --yes

# ── Database migrations ──
cd services/api
npx prisma migrate dev     # create + apply migration
npx prisma studio          # visual DB browser
```

13 Launch Phases

Staged rollout starting with Bangalore

0

Pre-Launch — Now

Landing page live. Collecting chef applications, nominations, and customer waitlist from Bangalore. Referral system driving organic word-of-mouth growth.

1

Phase 1 — Bangalore Pilot (5 Areas)

Koramangala, HSR Layout, Whitefield, Indiranagar, BTM Layout. Target: 20 verified home chefs, 200 customer orders. Full app on iOS + Android.

2

Phase 2 — Bangalore Scale (20 Areas)

Expand to 20 Bangalore areas. Target: 50 verified home chefs, optimize delivery logistics, introduce Cook's Kitchen Live + Feed a Neighbour at scale.

3

Phase 3 — Second City Expansion

Chennai, Hyderabad, or Pune based on waitlist demand. Replicate Bangalore playbook. Localize cuisine categories per city.

Maase India

connect.maase@gmail.com · maaseindia.com · Confidential — Internal Use Only

© 2026 Maase India · v1.0